

Desenvolupament d'aplicacions web

Programació en entorn servidor

UT1 – Introducció a llenguatges de servidor

Índex de continguts

1.1 Spring Core.....	3
1.2 Inversion of Control.....	4
1.3 Dependency Injection.....	6
1.3.1 DI per constructor.....	6
1.3.2 DI per setter.....	6
1.4 Beans.....	8
1.4.1 Definició.....	8
1.4.2 Scope.....	8
1.4.3 Lazy initialization.....	16
1.4.4 Cicle de vida d'un Bean.....	23

1.1 Spring Core

- **Spring Framework** és un conjunt de llibreries Java que faciliten la creació d'aplicacions.
- La seva base és **Spring Core**, que resol el problema de gestionar objectes i les seves dependències.
- És el **nucli del framework Spring**.
- Aporta les eines per gestionar els **objectes de l'aplicació** i les seves **dependències**.
- El principi clau és la **Inversió de Control (IoC)** i la **Injecció de Dependències (DI)**.
- Amb **Spring Boot**, arrencar projectes és més fàcil: evita configuracions manuals i ofereix auto-configuració.

En lloc de crear i gestionar manualment els objectes, és el **contenedor de Spring** qui ho fa.

1.2 Inversion of Control

En Java normal (POJO sense frameworks):

```
public class Engine {  
    public String start() {  
        return "Engine started!";  
    }  
}  
  
public class Car {  
    private Engine engine;  
  
    public Car() {  
        this.engine = new Engine(); // creació manual  
    }  
  
    public void drive() {  
        System.out.println(engine.start());  
    }  
}
```

Problemes:

- El Car depèn directament de Engine.

- Si volem canviar Engine (p. ex. ElectricEngine), hem de modificar el codi de Car.
- Díficil de testear: no podem injectar versions falses (MockEngine).

Solució: Inversió de Control (IoC)

Definició: principi de disseny on un objecte **no controla** la creació ni el cicle de vida de les seves dependències.

- En lloc de fer new i crear-les ell mateix, les **rep des de fora**.
- Amb Spring, aquest paper el fa el **contenedor IoC** (ApplicationContext), que:
 - Crea els objectes (beans).
 - Els configura i hi injecta dependències.
 - En gestiona el cicle de vida.

Benefici: menys acoblament, més flexibilitat i testabilitat.

1.3 Dependency Injection

Definició: tècnica mitjançant la qual un objecte (classe client) rep les seves dependències des del contenidor, en comptes de crear-les.

- Spring resol quins beans encaixen i els assigna.
- Evita dependències rígides i facilita substituir implementacions.

Exemple conceptual:

- Sense DI: *“El cotxe fabrica el motor.”*
- Amb DI: *“El taller (Spring) posa el motor al cotxe.”*

1.3.1 DI per constructor

- Les dependències es passen al moment de crear l'objecte.
- L'objecte queda **llest i complet** des de l'inici.
- Les dependències són **obligatòries** → millor seguretat i immutabilitat.
- És la forma **recomanada** a Spring.

1.3.2 DI per setter

- Les dependències s'injecten després de crear l'objecte, mitjançant mètodes `setXxx(...)`.
- Les dependències poden ser **opcionals**.
- L'objecte pot existir un temps sense tenir-ho tot configurat → més risc d'estat inconsistent.
- Útil en casos on:
 - Les dependències no són imprescindibles.

- Volem permetre canviar-les en temps d'execució.

Resum: Constructor → dependències obligatòries i immutables. Setter → dependències opcionals i mutables.



1.4 Beans

1.4.1 Definició

Definició: qualsevol objecte gestionat pel contenidor IoC de Spring.

- Per defecte, es poden declarar amb `@Component`.
- Però Spring ofereix **estereotips especialitzats**:
 - `@Component` → genèric (qualsevol classe).
 - `@Service` → classes de **lògica de negoci** (serveis de domini).
 - `@Repository` → classes d'**accés a dades**.
 - `@Controller` / `@RestController` → **presentació / web**.
- Es declara amb anotacions (`@Component`, `@Service`, etc.) o amb configuració (`@Bean`).
- El contenidor:
 - N'instancia un exemplar.
 - Li assigna les dependències.
 - En controla el cicle de vida segons l'**scope**.

En Spring Boot, el component scanning detecta automàticament les classes anotades i les converteix en beans.

1.4.2 Scope

Defineix **quantes instàncies d'un bean** crea Spring i **quan** es gestionen.

- Afecta al **cicle de vida** del bean dins del ApplicationContext.

Scopes més comuns:

Scope	Descripció	Vida del bean
singleton (per defecte)	Un únic bean per tot l'ApplicationContext. Totes les injeccions apunten a la mateixa instància.	Dura tant com el contenidor
prototype	Cada vegada que es demana el bean, Spring en crea una instància nova.	L'usuari n'és responsable; Spring no en controla la destrucció
request (web)	Un bean nou per a cada petició HTTP.	Dura el temps de la request
session (web)	Un bean per a cada sessió HTTP d'usuari.	Dura la sessió
application (web)	Un bean compartit per tota l'aplicació web (semblant a singleton, però a nivell del ServletContext).	Dura el context web

Quan fer servir cada scope?

singleton → serveis compartits, repositoris, caches, configuracions.

- **prototype** → objectes temporals, dades que canvien sovint, patrons on cal crear moltes instàncies independents.
- **request** → components que viuen només durant el processament d'una petició web.
- **session** → objectes que mantenen estat específic d'un usuari.
- **application** → dades o recursos compartits entre totes les sessions.

Exemples pràctics

Exemple A – Singleton (per defecte)

```
@Component

public class Contador {

    private AtomicInteger i = new AtomicInteger();

    public int next() {

        return i.incrementAndGet();

    }

}
```

```
@Service

public class ServiceA {

    private final Contador counter;

    @Autowired

    public ServiceA(Contador counter) {

        this.counter = counter;

    }

    public String print() {
```

```
        return "A: " + counter.next();  
    }  
  
}  
  
@Component  
public class ServiceB {  
  
    private final Contador counter;  
  
    @Autowired  
    public ServiceB(Contador counter) {  
        this.counter = counter;  
    }  
  
    public String print() {  
        return "B: " + counter.next();  
    }  
}
```

Tant ServiceA com ServiceB comparteixen **el mateix** Counter.
Executant A.print() i B.print(), els valors s'incrementen en continu.

Exemple B – Prototype

```
@Component  
  
@Scope("prototype")  
  
public class Ticket {  
  
    private static int seq = 0;  
  
    private int id;  
  
  
    public Ticket() {  
  
        id = seq++;  
  
    }  
  
  
    public int getId() {  
  
        return id;  
  
    }  
  
}
```

```
@Service

public class TicketService {

    private final Ticket ticket1;

    private final Ticket ticket2;

    @Autowired

    public TicketService(Ticket ticket1, Ticket ticket2){

        this.ticket1 = ticket1;

        this.ticket2 = ticket2;

    }

    public String printTicket(){

        return "Ticket1 = " + ticket1.getId() + " Ticket2 = " + ticket2.getId();

    }

}
```

- Amb prototype, ticket1 i ticket2 són instàncies diferents → IDs diferents.
- Si fos singleton, serien iguals.

Exemple C – Request

```
@Component

@Scope(value = "request", proxyMode =
ScopedProxyMode.TARGET_CLASS)

/*

    Empram un proxy ja que quan s'injecten les dependències (al controller)
    el bean encara no existeix

    ja que no hi ha cap request.

    Això introdueix un proxy en substitució del bean per a que es pugui injectar
    la dependència

    i quan arriba la request, ja es reemplaça per el bean que correspon.

*/

//@Scope(value="prototype")

public class RequestBean {

    private final String id = UUID.randomUUID().toString();


    public String getId() { return id; }

}

@Service

public class RequestService {
```

```
private final RequestBean requestBean;

@Autowired

public RequestService(RequestBean rb) {

    this.requestBean = rb;

}

public String getRequestBeanId() {

    return requestBean.getId();

}

}
```

Cada petició crea un RequestBean nou.

Exemple D – Session

```
@Component

@Scope(value="session", proxyMode =
ScopedProxyMode.TARGET_CLASS)

public class SessionBean {

    private final String id = UUID.randomUUID().toString();

    public String getId() { return id; }
```

```
}  
  
@Service  
  
public class SessionService {  
  
    private final SessionBean sessionBean;  
  
    @Autowired  
  
    public SessionService(SessionBean sessionBean) {  
  
        this.sessionBean = sessionBean;  
  
    }  
  
    public String getSessionBeanId() {  
  
        return sessionBean.getId();  
  
    }  
  
}
```

Ara el bean es manté mentre es mantingui la sessió HTTP. Per exemple, ho podem provar obrint un navegador diferent i enviant-li una petició al controller.

1.4.3 Lazy initialization

Per defecte, **Spring crea (eager)** tots els **beans singleton** detectats durant l'arrencada del `ApplicationContext` (component scan + `@Configuration` + `@Bean`). Això fa que el **temps d'arrencada** sigui més llarg però garanteix que **els errors de wiring** apareguin de seguida.

Amb **lazy initialization**, **ajornem la creació** d'alguns beans fins **al primer ús real**.

Què vol dir “lazy” exactament?

- **Eager (per defecte, singleton):** el bean es **construeix** i s’inicialitza quan arrenca l’aplicació.
- **Lazy:** el bean **no es crea** a l’arrencada; s’injecta **un proxy** i la instància real es crea **quan s’accedeix per primer cop**.

Nota: els **beans prototype** ja són “on demand” per definició (es creen quan es demanen). Marcar-los @LAZY no aporta res.

Maneres d’activar lazy

1. **Al nivell de classe/bean:** @Lazy sobre la classe @Component o el mètode @Bean.
2. **Al punt d’injecció:** @Lazy sobre el paràmetre/atribut injectat (només aquella dependència es resol tard).
3. **Global (Spring Boot):** `spring.main.lazy-initialization=true` (fa lazy per a tots els beans, excepte casos especials).

Exemple 1 — Eager (per defecte)

Objectiu: veure un bean “pesat” creant-se a l’arrencada.

```
// Bean pesat

@Component

public class HeavyBean {

    public HeavyBean() {
```

```
System.out.println(">> Constructor HeavyBean: inicialitzant (5s)...");

try {

    Thread.sleep(5000); // simula feina pesada

} catch (InterruptedException ignored) {}

}

public String process() {

    return "HeavyBean process completat!";

}

}

@Service

public class HeavyService {

    private final HeavyBean heavyBean;

    @Autowired

    public HeavyService(HeavyBean heavyBean) {

        this.heavyBean = heavyBean;

    }

}
```

```
}  
  
public String doWork() {  
    return heavyBean.process();  
}  
}
```

Què passa?

Quan arrenca Spring, es crea HeavyService (5s), **abans** d'executar cap lògica de l'app.

Exemple 2 — Lazy al bean i punt d'injecció

Objectiu: que HeavyService **no** es creï fins que realment l'usem.

```
@Component  
  
@Lazy  
  
public class HeavyBean {  
  
    public HeavyBean() {
```

```
System.out.println(">> Constructor HeavyBean: inicialitzant (5s...)");

try {

    Thread.sleep(5000); // simula feina pesada

} catch (InterruptedException ignored) {}

}

public String process() {

    return "HeavyBean process completat!";

}

}

@Service

public class HeavyService {

    private final HeavyBean heavyBean;

    @Autowired

    public HeavyService(@Lazy HeavyBean heavyBean) {

        this.heavyBean = heavyBean;

    }

}
```

```
public String doWork() {  
  
    return heavyBean.process();  
  
}  
  
}
```

Perquè tant al Bean com al punt d'injecció?

- @Lazy al bean (@Component)
 - Marca el bean com a “crea'm només quan em demanin explícitament”.
 - Però... si un singleton (com HeavyService) el declara al seu constructor, Spring necessita tenir una instància per satisfer aquella dependència en arrencar.
 - Per tant, Spring l'acaba creant igualment i el “lazy” queda anul·lat.
- @Lazy al punt d'injecció
 - Indica a Spring que no injecti directament el bean real, sinó un proxy.
 - El proxy és un placeholder que es pot injectar immediatament en el singleton.
 - Quan invoques un mètode del proxy (process()), aleshores es crea el bean real.
 - Aquí sí que tens lazy... però només si el bean en si també admet retardar la seva creació.
- Quan poses @Lazy a tots dos llocs
 - Al bean → dius a Spring: “aquest no el creïs fins que sigui necessari”.

- A l'injecció → dius a Spring: "aquí injecta'm un proxy i no l'instanciïs ara".
- La combinació és la que força Spring a no crear-lo a l'arrencada, sinó al primer ús real.

Exemple 3 — Lazy global (Spring Boot)

A src/main/resources/application.properties:

```
spring.main.lazy-initialization=true
```

Efecte: tots els beans passen a lazy, excepte els crítics per arrencada/infraestructura.

Per **forçar eager** en un bean concret:

```
@Lazy(false)

@Component

public class Warmup { /* ... */ }
```

Quan (no) fer-lo servir

Bons usos

- Beans costosos (inicialitzacions llargues, connexions diferides) **no imprescindibles** a l'arrencada.
- Dependre d'un component ocasional (p. ex. un client extern que no sempre s'usa).

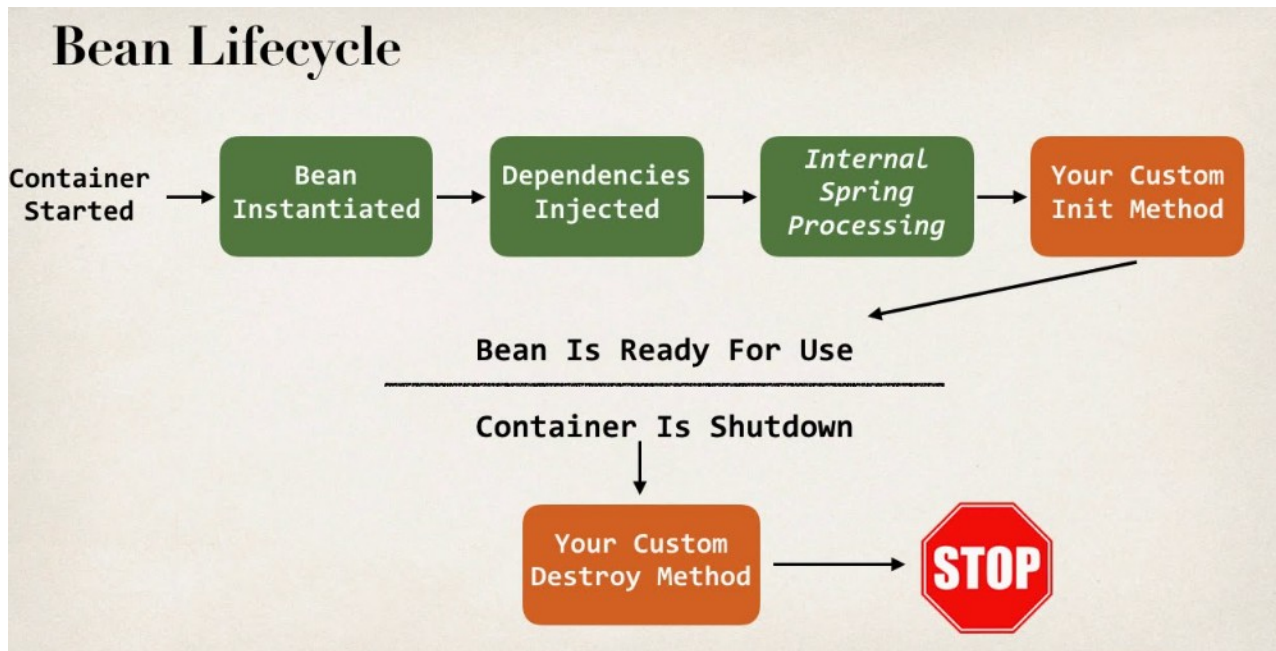
Riscos / trade-offs

- **Errors tardans:** si hi ha problemes de wiring/config, apareixeran **al primer ús**, no a l'arrencada.
- **Cold start** de la primera petició: la **primera crida** que accedeix al bean pot patir **latència extra**.
- Si hi ha **concurrència** en el primer ús, el proxy s'encarrega d'inicialitzar una sola vegada, però pots notar bloqueig inicial.

Cheatsheet ràpid

- Per defecte, **singleton = eager**.
- @Lazy sobre **classe** o **@Bean** → el bean es crea en **primer ús**.
- @Lazy sobre **paràmetre/atribut** → només aquella **dependència** es resol tard.
- **Global:** spring.main.lazy-initialization=true (i després @Lazy(false) per excepcions).
- **Prototype:** ja és "on demand"; @Lazy no aporta res.

1.4.4 Cicle de vida d'un Bean



1. **Instanciació:** el contenidor crea l'objecte.
2. **Injecció de dependències:** es resolen i assignen tots els beans necessaris.
3. **Inicialització:**
 - Es poden executar mètodes especials (`@PostConstruct`).
 - També passen pels `BeanPostProcessor` si n'hi ha.
4. **Ús:** el bean viu dins del contenidor i l'aplicació el pot utilitzar.
5. **Destrucció:**
 - En tancar l'aplicació, Spring crida mètodes de finalització (`@PreDestroy`) si escau.
 - En el cas de beans `prototype`, Spring no n'administra la destrucció.

Aquest cicle permet a Spring gestionar recursos de forma controlada i centralitzada.

Anem amb un exemple per veure el comportament.

```
@Component

public class ReportGenerator {

    public ReportGenerator() {

        System.out.println(">> Constructor ReportGenerator");

    }

    @PostConstruct

    public void init() {

        System.out.println(">> Inicialitzant recursos del ReportGenerator");

    }

    @PreDestroy

    public void cleanup() {

        System.out.println(">> Alliberant recursos del ReportGenerator");

    }

}
```

```
public String generate() {  
    return "Informe generat!";  
}  
  
}  
  
@Service  
public class ReportService {  
  
    private final ReportGenerator reportGenerator;  
  
    @Autowired  
    public ReportService(ReportGenerator reportGenerator) {  
        this.reportGenerator = reportGenerator;  
    }  
  
    public String createReport() {  
        return reportGenerator.generate();  
    }  
}
```

Què passa en executar?

1. Arrencada
 - i. Spring crea el ReportGenerator (constructor).
 - ii. Executa `@PostConstruct`.
 - iii. El bean queda llest per ser usat.
2. En cridar /report
 - i. El ReportController usa el ReportGenerator.
 - ii. Es retorna el text "Informe generat!".
3. En tancar l'aplicació
 - i. Spring crida `@PreDestroy` per alliberar recursos.

Casos d'ús habituals dels hooks:

- **`@PostConstruct`**
 - Carregar dades a memòria en arrencar (cache inicial).
 - Configurar connexions externes (clients HTTP, SDKs de tercers).
 - Obrir connexions a un broker de missatges (Kafka, RabbitMQ).
 - Inicialitzar un planificador de tasques (ScheduledExecutorService).
 - Validar valors de configuració i fallar ràpid si hi ha errors.

- Preparar recursos que depenen de dependències injectades (fitxers, plantilles, objectes derivats).
- **@PreDestroy**
 - Tancar connexions a BBDD, sockets o clients HTTP.
 - Aturar executors o pools de threads.
 - Cancel·lar tasques planificades.
 - Alliberar recursos natius (fitxers temporals, handles del sistema).
 - Persistir estat temporal abans de tancar l'aplicació.
 - Desregistrar-se d'un servei extern (per ex. tancar subscripcions a un event bus).